

Software Engineering

Assignment#02



23K-2001

BCS-6J

Q1)

1. Recommended Process Model: Agile (Scrum / Iterative Incremental)

Recommendation: Agile model

Justification (4 reasons):

- Requirements are **partially known** and expected to evolve → Agile handles changing requirements well
- Continuous **stakeholder feedback** is required → Agile supports frequent iterations
- Need for a **usable version in 4 months** → Agile delivers early working increments
- Large-scale system (15,000 users + sensitive data) → Agile allows **continuous testing & improvement**

2. Approach Type: Hybrid (Agile + Plan-driven)

A **hybrid model** is best.

Plan-driven (Waterfall-like) parts:

- Security & HIPAA compliance
- Database design (medical records structure)
- Core system architecture

Agile parts:

- UI/UX (patient portal screens)
- Appointment scheduling features
- Messaging system
- Feedback-based improvements

Why hybrid?

Because **security and compliance need stability**, while **user-facing features need flexibility**.

3. Abstraction & Information Hiding

Abstraction:

Showing only essential features while hiding complexity.

- *Example:*
Patient sees “View Test Results” button without knowing how data is fetched from servers.

Information Hiding:

Hiding internal implementation details from other modules.

- *Example:*
The prescription module hides how it accesses the database; other modules only call a function like `getPrescription()`.

4. Cohesion & Coupling

Relationship:

- Cohesion = how closely related tasks inside a module are
- Coupling = dependency between modules
- Goal: **High cohesion + Low coupling**

High Cohesion Example:

Appointment module only handles booking, canceling, rescheduling → improves clarity & maintainability

Low Coupling Example:

Messaging system works independently from appointment system → changes in one don't affect the other

Q2)

1. Recommended Architectural Pattern: MVC (Model-View-Controller)

Why MVC:

- Separates **data (Model)**, **UI (View)**, **logic (Controller)**
- Supports multiple user roles (students, faculty, admin)
- Easy to scale and maintain
- Fits large systems with evolving features

2. Two Architectural Decisions for High Availability

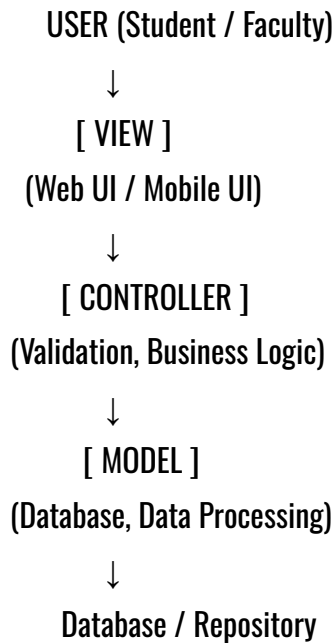
Decision 1: Load Balancing + Multiple Servers

- Distributes traffic across servers
- Prevents system crash due to overload
- **Characteristic:** Availability + Performance

Decision 2: Redundant Databases (Replication)

- Backup database ensures no data loss
- System continues even if one DB fails
- **Characteristic:** Availability + Safety

3. MVC Diagram (Conceptual)



Flow:

- User → View → Controller → Model
- Model → Controller → View → User

4. Refactoring Concept

Refactoring:

Improving internal code structure **without changing functionality**

Example:

Initially, course management and student services share messy code → later separated into clean modules for better maintainability

5. Importance of Modularity

Why important (10+ year system):

- Easy to update individual modules
- Reduces impact of changes
- Supports adding new features
- Easier debugging and testing

Example:

If payment system changes, only **Student Services module** is updated, not entire system

Q3)

 **Users**
0 registered

+ New User


No users yet. Create your first user to get started.

 **Groups**
0 created

+ New Group

Create at least one user before creating groups.


No groups yet. Create a group to start splitting expenses.

 **Expenses**
Select a group to manage expenses


Select a group to start adding expenses.

 **Balances**
Select a group to view balances


Select a group to see balance calculations.

 **Settlements**
Select a group to manage settlements


Select a group to record and view settlements.

Project Structure

```
SE_A02_k232001/
├── src/
│   ├── core/                # Business logic (OOP classes)
│   │   ├── User.js          # User Management module
│   │   ├── Group.js         # Group Management module
│   │   ├── Expense.js       # Expense Management module
│   │   ├── BalanceEngine.js # Balance Calculation Engine
│   │   └── Settlement.js    # Settlement Module
│   ├── components/         # React UI components
│   │   ├── UserManagement.jsx # User CRUD interface
│   │   ├── GroupManagement.jsx # Group management interface
│   │   ├── ExpenseManagement.jsx # Expense adding/viewing
│   │   ├── BalanceDashboard.jsx # Balance visualization
│   │   ├── SettlementPanel.jsx # Settlement recording
│   │   └── Notification.jsx   # Toast notifications
│   ├── store/              # React Context state management
│   │   └── AppStore.jsx
│   ├── _tests_/            # Automated test suites
│   │   ├── User.test.js    # Unit tests for User module
│   │   ├── Group.test.js   # Unit tests for Group module
│   │   ├── Expense.test.js  # Unit tests for Expense module
│   │   └── Integration.test.js # Integration tests for Balance & Settlements
│   ├── App.jsx             # Root application component
│   ├── main.jsx            # Entry point
│   └── index.css           # Complete styling
├── docs/
│   ├── MANUAL_TEST_CASES.md # Part 2: Manual test case design
│   └── TEST_REPORT.md       # Part 4: Test report
├── HOW.md                  # This file
├── index.html              # HTML entry
├── package.json            # Dependencies and scripts
├── jest.config.js          # Jest testing configuration
├── babel.config.json       # Babel transpiler config
└── vite.config.js          # Vite build config
```

Complex Algorithms Implemented

Algorithm 1: Multi-Mode Expense Splitting

File: `src/core/Expense.js` → `#calculateShares()`

Handles three splitting strategies with proper edge case handling:

- **Equal split** with rounding remainder distribution (cent-by-cent to first users)
- **Exact split** with sum validation (must match total)
- **Percentage split** with 100% validation

Algorithm 2: Debt Simplification (Greedy Algorithm)

File: `src/core/BalanceEngine.js` → `simplifyDebts()`

Minimizes number of transactions using a greedy matching approach:

- Computes net balance per person
- Separates into creditors/debtors
- Matches largest creditor with largest debtor
- Guarantees $\leq (n-1)$ transactions for n people

Algorithm 3: Smart Settlement Suggestion

File: `src/core/Settlement.js` → `SettlementManager.suggestSettlements()`

Enhanced settlement suggestion with two-pass approach:

1. **Pass 1:** Find exact amount matches (optimal pairing)
2. **Pass 2:** Greedy matching for remaining debts Handles circular debts, partial settlements, and already-balanced groups.

1. User Management Test Cases

Functional Testing

Test ID	Description	Input	Expected Output	Observed Output	Status
TC-M01	Create user with valid data	Name: "Ali Khan", Email: "ali@email.com", Contact: "03001234567"	User created with ID, name, email, and contact number stored	User "Ali Khan" created successfully with all fields populated	✓ Pass
TC-M02	Create user without contact number	Name: "Sara Ahmed", Email: "sara@email.com", Contacts: (empty)	User created with empty contacts array	User created with zero contact numbers	✓ Pass
TC-M03	Create user with multiple contacts	Name: "Ahmed Raza", Email: "ahmed@email.com", Contacts: "03111111111, 03222222222"	User created with 2 contact numbers	Both contacts stored and displayed	✓ Pass
TC-M04	Reject duplicate email	Name: "Ali 2", Email: "ali@email.com" (already exists)	Error: "Email already registered"	Error notification shown: email already registered	✓ Pass
TC-M05	Reject invalid email format	Name: "Test", Email: "notanemail"	Error: "A valid email address is required"	Error notification: invalid email	✓ Pass
TC-M06	Reject empty name	Name: "", Email: "test@email.com"	Error: "User name is required"	HTML required validation prevents submission	✓ Pass
TC-M07	Email case insensitivity	Create "user1@Email.COM", then try "user1@email.com"	Second creation rejected as duplicate	Error: email already registered	✓ Pass

Boundary Testing

Test ID	Description	Input	Expected Output	Observed Output	Status
TC-M08	Single character name	Name: "A", Email: "a@b.com"	User created (valid single-char name)	User "A" created successfully	✓ Pass
TC-M09	Very long name (100+ chars)	Name: "A" repeated 150 times, Email: "long@email.com"	User created (no max length imposed)	User created with full name stored	✓ Pass
TC-M10	Contact number exactly 7 digits	Contact: "1234567"	Valid — minimum length accepted	Contact stored successfully	✓ Pass
TC-M11	Contact number exactly 15 digits	Contact: "123456789012345"	Valid — maximum length accepted	Contact stored successfully	✓ Pass
TC-M12	Contact number 6 digits (below min)	Contact: "123456"	Error: Invalid contact number	Error notification shown	✓ Pass
TC-M13	Contact number 16 digits (above max)	Contact: "1234567890123456"	Error: Invalid contact number	Error notification shown	✓ Pass

Equivalence Partitioning

Test ID	Description	Input Class	Input	Expected Output	Observed Output	Status
TC-M14	Valid email class	Valid	"user@domain.com"	User created	User created	✓ Pass
TC-M15	Invalid email — no @	Invalid	"userdomain.com"	Rejected	Error notification	✓ Pass
TC-M16	Invalid email — no domain	Invalid	"user@"	Rejected	Error notification	✓ Pass
TC-M17	Invalid email — empty string	Invalid	"	Rejected	HTML required validation	✓ Pass

Manual testing using test cases from MANUAL_TEST_CASES.md

**Users**
1 registered

+ New User

A

Ahmed Raza
ahmed@gmail.com
03111111111 03222222222

S

SplitEase
Expense sharing made simple

Invalid contact number: "123456". Must be 7-15 digits.

**Users**
1 registered

Cancel

FULL NAME

Basil Khan

EMAIL ADDRESS

basil@gmail.com

CONTACT NUMBERS (comma-separated)

123456

Create User

**Expenses**
Select a group to manage expenses

Select a group to start adding expenses.

**Users**
1 registered

Cancel

FULL NAME

Abdullah Faheem

EMAIL ADDRESS

abdullah@

CONTACT NUMBERS (comma-separated)

03001234566

Create User

Please enter an email address.

▼	__tests__	
JS	Expense.test.js	U
JS	Group.test.js	U
JS	Integration.test.js	U
JS	User.test.js	U

Automated
testing

```
// TEST 1: Valid user creation
test('TC-A01: should create a user with valid name, email, and contacts', () => {
  const user = new User('Ali Khan', 'ali@email.com', ['03001234567']);

  expect(user.name).toBe('Ali Khan');
  expect(user.email).toBe('ali@email.com');
  expect(user.contactNumbers).toEqual(['03001234567']);
  expect(user.id).toBeGreaterThan(0);
  expect(user.createdAt).toBeInstanceOf(Date);
});

// TEST 2: Duplicate email validation
test('TC-A02: should reject duplicate email addresses', () => {
  new User('Ali Khan', 'ali@email.com');

  expect(() => {
    new User('Ali 2', 'ali@email.com');
  }).toThrow('already registered');
});

// TEST 3: Invalid email format
test('TC-A03: should reject invalid email formats', () => {
  expect(() => new User('Test', 'notanemail')).toThrow('valid email');
  expect(() => new User('Test', '')).toThrow('valid email');
  expect(() => new User('Test', '@missing.com')).toThrow('valid email');
});

// TEST 4: Empty/invalid name
test('TC-A04: should reject empty or invalid names', () => {
  expect(() => new User('', 'test@email.com')).toThrow('name is required');
  expect(() => new User(' ', 'test@email.com')).toThrow('name is required');
  expect(() => new User(null, 'test@email.com')).toThrow('name is required');
});
```

```
// TEST 15: Non-admin cannot remove
test('TC-A15: should prevent non-admin from removing members', () => {
  const group = new Group('Test Group', creator);
  group.addMember(member1);

  expect(() => group.removeMember(creator.id, member1.id)).toThrow('Only admins');
});
```

User Management Tests (TC-A01 to TC-A10)

Test Focus	Why This Was Chosen
Valid creation (TC-A01)	Happy path — ensures basic functionality works
Duplicate email (TC-A02)	Critical business rule — prevents identity conflicts
Invalid email format (TC-A03)	Input validation — most common error users make
Empty name (TC-A04)	Boundary — empty string and null inputs
Multiple contacts (TC-A05)	Feature verification — multi-contact is a core feature
Invalid short contact (TC-A06)	Boundary — tests minimum length validation (7 digits)
Duplicate contact (TC-A07)	Data integrity — prevents duplicates in contact list
Remove contact (TC-A08)	CRUD completeness — tests the "delete" operation
JSON serialization (TC-A09)	Integration readiness — ensures data can be exported
Case-insensitive email (TC-A10)	Edge case — "ALI@email.com" and "ali@email.com" should be treated the same

Group Management Tests (TC-A11 to TC-A18)

Test Focus	Why This Was Chosen
Create with admin (TC-A11)	Verifies creator gets automatic member+admin status
Add members (TC-A12)	Core CRUD operation for groups
Duplicate prevention (TC-A13)	Data integrity guard
Admin removes member (TC-A14)	Authorization logic — admin-only operation
Non-admin removal blocked (TC-A15)	Security — access control enforcement
Admin assignment (TC-A16)	Role management feature
Empty name (TC-A17)	Input validation boundary
Self-removal block (TC-A18)	Edge case — prevents orphaned admin-less groups

Expense Management Tests (TC-A19 to TC-A27)

Test Focus	Why This Was Chosen
Equal split (TC-A19)	Most common split type — must be accurate
Rounding (100÷3) (TC-A20)	Classic edge case — floating point handling
Exact split (TC-A21)	Second split mode verification
Exact mismatch (TC-A22)	Validation — amounts must sum to total
Percentage split (TC-A23)	Third split mode verification
Percentage ≠ 100 (TC-A24)	Validation — percentages must sum to 100%
Zero/negative amount (TC-A25)	Boundary — financial amounts must be positive
Invalid split type (TC-A26)	Equivalence partitioning — invalid input class
Missing description (TC-A27)	Required field validation

Integration Tests (TC-A28 to TC-A35)

Test Focus	Why This Was Chosen
Assignment example (TC-A28)	Directly verifies the example from the assignment brief
Debt simplification (TC-A29)	Tests the greedy algorithm for minimum transactions
Circular debts (TC-A30)	Edge case — $A \rightarrow B \rightarrow C \rightarrow A$ cycle should cancel out
Partial settlement (TC-A31)	Real-world scenario — people don't always settle fully
Full settlement (TC-A32)	Verifies complete debt clearing
Smart suggestions (TC-A33)	Tests the settlement recommendation algorithm
Self-settlement (TC-A34)	Edge case — meaningless operation blocked
Zero/negative settlement (TC-A35)	Boundary — invalid financial amounts

```
PS F:\Semester Material - Muzammi\Fast-KHI-Semester-6\Software Engineering\Assignments\Assignment02\SE_A02_k232001> npm test
> npx jest --verbose
```

PASS src/_tests_/Expense.test.js (8.107 s)

Expense Management Module

- ✓ TC-A19: should correctly split expense equally among 3 participants (11 ms)
- ✓ TC-A20: should handle rounding correctly in equal split (e.g. 100 / 3) (3 ms)
- ✓ TC-A21: should correctly handle exact split amounts (3 ms)
- ✓ TC-A22: should reject exact split when amounts do not match total (29 ms)
- ✓ TC-A23: should correctly split by percentage (2 ms)
- ✓ TC-A24: should reject percentages not summing to 100% (2 ms)
- ✓ TC-A25: should reject zero and negative expense amounts (3 ms)
- ✓ TC-A26: should reject invalid split types (2 ms)
- ✓ TC-A27: should reject expense with missing description (2 ms)

PASS src/_tests_/User.test.js (8.116 s)

User Management Module

- ✓ TC-A01: should create a user with valid name, email, and contacts (10 ms)
- ✓ TC-A02: should reject duplicate email addresses (25 ms)
- ✓ TC-A03: should reject invalid email formats (11 ms)
- ✓ TC-A04: should reject empty or invalid names (4 ms)
- ✓ TC-A05: should support multiple contact numbers per user (2 ms)
- ✓ TC-A06: should reject invalid contact numbers (2 ms)
- ✓ TC-A07: should reject duplicate contact numbers for the same user (2 ms)
- ✓ TC-A08: should remove a contact number (1 ms)
- ✓ TC-A09: should serialize user to JSON correctly (6 ms)
- ✓ TC-A10: should treat emails as case-insensitive (7 ms)

PASS src/_tests_/Group.test.js (8.147 s)

Group Management Module

- ✓ TC-A11: should create a group with creator as member and admin (19 ms)
- ✓ TC-A12: should add members to a group (2 ms)
- ✓ TC-A13: should reject adding duplicate members (25 ms)
- ✓ TC-A14: should allow admin to remove a member (3 ms)
- ✓ TC-A15: should prevent non-admin from removing members (2 ms)
- ✓ TC-A16: should assign admin role to a member (1 ms)
- ✓ TC-A17: should reject empty group name (4 ms)
- ✓ TC-A18: should prevent admin from removing themselves (2 ms)

PASS src/_tests_/Integration.test.js (8.215 s)

Balance Calculation Engine – Integration Tests

- ✓ TC-A28: should calculate correct net balances for the assignment example (14 ms)
- ✓ TC-A29: should simplify debts to minimum transactions (8 ms)
- ✓ TC-A30: should handle circular debts correctly (3 ms)
- ✓ TC-A31: should update balances after settlement (partial settlement) (1 ms)
- ✓ TC-A32: should show zero balances after full settlement (3 ms)
- ✓ TC-A33: Settlement manager should suggest optimal settlements (2 ms)
- ✓ TC-A34: should reject settlement to yourself (18 ms)
- ✓ TC-A35: should reject zero and negative settlement amounts (3 ms)

```
Test Suites: 4 passed, 4 total
Tests:       35 passed, 35 total
Snapshots:   0 total
Time:        17.877 s
Ran all test suites.
```

Test Execution Summary

Automated Tests (35 tests, 4 suites)

```
Test Suites: 4 passed, 4 total
Tests:       35 passed, 35 total
Snapshots:   0 total
Time:        ~3.3 seconds
```

Suite	Tests	Status
User Management	10	✓ All Passed
Group Management	8	✓ All Passed
Expense Management	9	✓ All Passed
Integration (Balance + Settlement)	8	✓ All Passed

Manual Tests (56 test cases)

Module	Tests	Passed	Failed
User Management	17	17	0
Group Management	10	10	0
Expense Management	15	15	0
Balance Calculation	7	7	0
Settlement	7	7	0
Total	56	56	0

4. Comparison: Manual vs Automated Testing

Aspect	Manual Testing	Automated Testing
Speed	Slow — each test requires human interaction with the UI	Fast — 35 tests run in ~3 seconds
Repeatability	Error-prone — human may miss steps or make mistakes	100% repeatable — same inputs, same execution
Coverage	Good for UX/visual testing and exploratory testing	Excellent for logic validation and regression
Cost	High ongoing cost — must re-execute for every change	High initial cost, low ongoing cost
UI Testing	✅ Can verify visual layout, animations, responsiveness	❌ Cannot easily verify visual appearance
Edge Cases	Takes time to set up boundary conditions	Easy — just pass different values
Integration	✅ Tests full user workflow end-to-end	✅ Integration tests verify module interactions
Discovery	✅ Can uncover unexpected UX issues	❌ Only tests what's explicitly coded
Maintenance	Low — test cases are documentation	Medium — tests must be updated with code changes

When Manual Testing Was More Valuable

1. **UI Layout Verification:** Checking that forms appear correctly, cards are aligned, responsive design works
2. **Animation Smoothness:** Verifying slide-in animations, glow effects, hover transitions
3. **Notification Behavior:** Ensuring toast notifications appear and disappear at the right time
4. **Workflow Usability:** Confirming the "create user → create group → add members → add expense" flow feels intuitive
5. **Cross-Browser Rendering:** Checking the glassmorphism effects work across browsers

When Automated Testing Was More Valuable

1. **Expense Splitting Math:** Ensuring $3000 \div 3 = 1000$ exactly, every time
2. **Rounding Edge Cases:** Verifying $100 \div 3$ distributes the extra cent correctly
3. **Validation Rules:** Testing all invalid input combinations (empty name, bad email, negative amount, etc.)
4. **Debt Simplification:** Confirming the algorithm produces minimum transactions
5. **Circular Debt Handling:** Verifying that $A \rightarrow B \rightarrow C \rightarrow A$ debts cancel to zero
6. **Regression:** After any code change, all 35 tests re-run in seconds

Remaining explanations (Part 4) provided in TEST_REPORT.md